

Project Title: AI-Enhanced Code & Flowchart Learning System

Project Type	Full-stack Educational Learning Platform with ML and GenAI
Frontend	React, TypeScript, Vite, Tailwind CSS
Backend	FastAPI, SQLAlchemy, JWT Authentication
Database	SQLite
Machine Learning	Decision Tree Classifier
Generative AI	Gemini API
Deployment	Vercel + FastAPI
Domain	Programming Education
Version	1.0

Problem Statement

Programming students often struggle to understand source code, logic flow, algorithm behavior, and program execution. AI-Enhanced Code & Flowchart Learning System addresses this issue by providing visual flowcharts, code generation, machine learning-based logic classification, and AI-powered algorithm explanations.

Approach

The system follows a full-stack approach integrated with Machine Learning and Generative AI. User code or flowcharts are processed by the backend, analyzed using a Decision Tree model, and enhanced with Gemini-powered explanations and generation services.

System Architecture

User → React Frontend → FastAPI Backend → SQLite Database / Decision Tree Model / Gemini API → Generated Results

Tools Used

React, TypeScript, Vite, Tailwind CSS, FastAPI, SQLite, Scikit-Learn, Decision Tree Classifier, Gemini API, Git, GitHub, Vercel, VS Code

Objectives

Develop a complete learning platform, convert code into flowcharts, generate code from flowcharts, classify programming logic using a Decision Tree model, and provide AI-generated explanations to improve understanding.

Scope of the System

The system supports user authentication, dashboard analytics, code-to-flowchart conversion, flowchart-to-code generation, logic classification, algorithm explanation, session history, user profile management, and settings.

Project Folder Structure

Frontend contains React UI components and pages. Backend contains FastAPI routes, services, authentication, and database logic. ML contains dataset, notebook, training script, and saved model. Documentation contains project reports.

Technology Stack

Frontend: React, TypeScript, Vite, Tailwind CSS. Backend: FastAPI, SQLAlchemy, JWT. Database: SQLite. Machine Learning: Scikit-Learn Decision Tree. AI: Gemini API.

System Architecture

Users interact with the React frontend. Requests are sent to the FastAPI backend. The backend accesses the database, machine learning services, and Gemini AI services. Results are returned to the frontend for display.

Backend Design

Main modules include authentication, database configuration, API routes, machine learning integration, flowchart services, and Gemini-powered explanation services.

Database Design

The database stores users, session history, generated flowcharts, algorithm explanation records, and logic classification records. Relationships ensure efficient retrieval of learning history.

Frontend Design

The frontend includes Dashboard, Code-to-Flowchart, Flowchart-to-Code, Logic Classification, Algorithm Explanations, History, Profile, and Settings pages. Responsive UI ensures accessibility.

Machine Learning Module

A Decision Tree Classifier is trained using programming logic features such as conditions, loops, functions, operators, code length, and complexity score. The model predicts logic categories including Sequential, Conditional, Looping, Nested, and Function-Based Logic.

Generative AI Module

Gemini API is used for algorithm explanations, code understanding, and educational assistance. The AI provides summaries, logic descriptions, complexity analysis, and step-by-step explanations.

System Integration

Frontend, backend, machine learning, database, and Gemini services are fully integrated. APIs connect all modules into a single learning platform.

Important API Endpoints

Authentication APIs handle login and registration. Flowchart APIs perform conversions. Classification APIs predict logic type. Explanation APIs generate AI-powered algorithm explanations.

Setup Instructions

Install backend dependencies using pip. Run FastAPI using Uvicorn. Install frontend dependencies using npm and run the Vite development server. Configure environment variables before execution.

Environment Configuration

GEMINI_API_KEY stores the Gemini API key. DATABASE_URL defines the database location. JWT_SECRET_KEY secures authentication tokens.

Deliverables

Frontend source code, backend source code, ML dataset, notebook, trained model, documentation, GitHub repository, deployment links, and demonstration video.

Expected Outcome

The project demonstrates how Artificial Intelligence, Machine Learning, and Full Stack Development can improve programming education through visualization, automation, and intelligent explanations.

Technology Stack Table

Frontend	React, TypeScript, Tailwind CSS
Backend	FastAPI
Database	SQLite
ML	Scikit-Learn
AI	Gemini API
Deployment	Vercel

Important Modules

Dashboard	Overview of platform
Code to Flowchart	Visualize code
Flowchart to Code	Generate source code
Logic Classification	Predict logic type
Algorithm Explanation	AI explanations
History	Store sessions
Profile	User details
Settings	Preferences